

aula_pratica_agrupamento_kmeans_dbscan

June 16, 2026

0.1 1. Introdução

Em problemas de **classificação**, normalmente temos exemplos com rótulos conhecidos.

Exemplo:

Idade	Renda	Classe
22	1800	Baixo risco
45	7000	Alto risco

Já em problemas de **agrupamento**, os dados não possuem rótulos. O objetivo é encontrar estruturas ou grupos naturais nos dados.

Exemplo:

Idade	Renda
22	1800
45	7000
30	2500
50	9000

O algoritmo tenta responder perguntas como:

- Existem grupos parecidos?
- Quais pontos são semelhantes entre si?
- Há pontos muito diferentes dos demais?
- Os dados parecem formar grupos separados?

Nesta aula vamos trabalhar com dois algoritmos:

1. **K-Means**: agrupa os pontos com base na distância até centros chamados centroides.
2. **DBSCAN**: agrupa pontos com base em regiões de alta densidade e consegue identificar ruídos/outliers.

```
[19]: # Importando as bibliotecas necessárias
```

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

```

from sklearn.datasets import make_blobs, make_moons
from sklearn.cluster import KMeans, DBSCAN
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, adjusted_rand_score
from sklearn.neighbors import NearestNeighbors

np.random.seed(42)

```

0.2 2. Criando uma função auxiliar para visualização

Para evitar repetir código, criaremos uma função para mostrar os pontos em um gráfico 2D.

A função pode mostrar os pontos sem agrupamento ou coloridos de acordo com os grupos encontrados por algum algoritmo.

```

[20]: def plot_clusters(X, labels=None, centroids=None, title="Visualização dos
      ↪ dados"):
    plt.figure(figsize=(7, 5))

    if labels is None:
        plt.scatter(X[:, 0], X[:, 1], s=40)
    else:
        plt.scatter(X[:, 0], X[:, 1], c=labels, s=40)

    if centroids is not None:
        plt.scatter(
            centroids[:, 0],
            centroids[:, 1],
            marker="X",
            s=250,
            label="Centroides"
        )
    plt.legend()

    plt.title(title)
    plt.xlabel("Atributo 1")
    plt.ylabel("Atributo 2")
    plt.grid(True)
    plt.show()

```

1 Parte 1 — K-Means

1.1 3. Criando um conjunto de dados simples

Primeiro, vamos criar dados artificiais com três grupos bem separados.

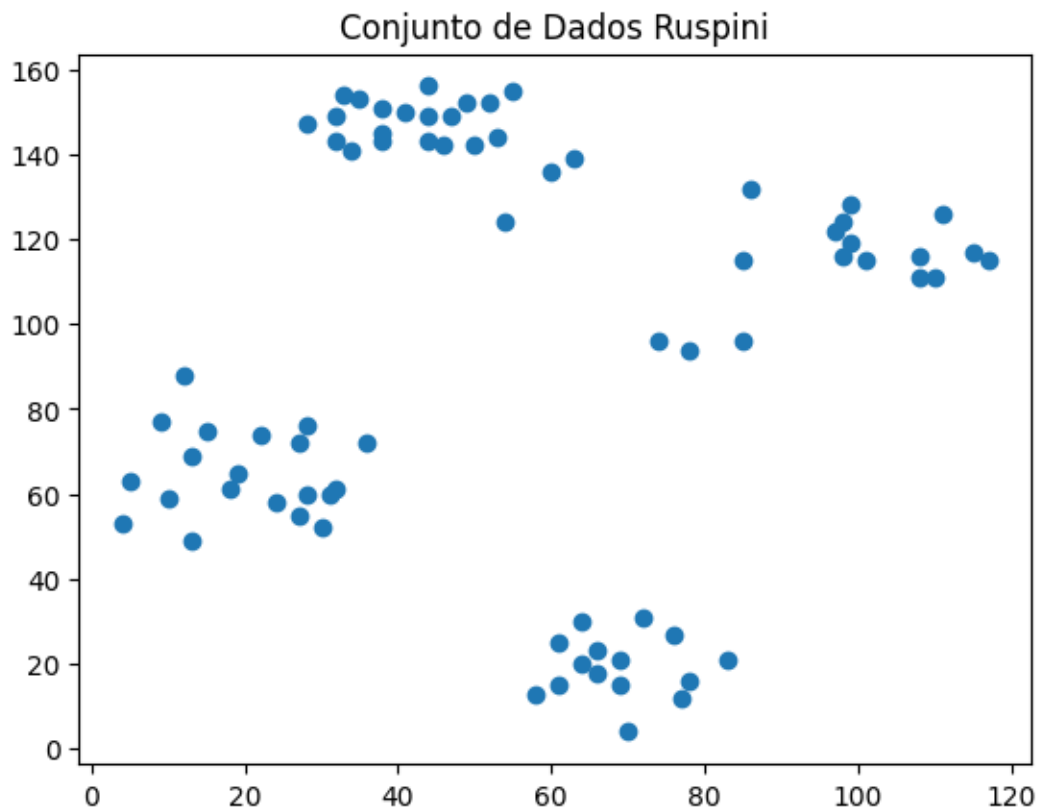
Esse tipo de dado favorece o funcionamento do K-Means, porque os grupos têm formato aproximadamente circular e estão relativamente separados.

```
[ ]: df = pd.read_csv('ruspini.csv')

X = df[['x', 'y']].values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

plt.scatter(X[:, 0], X[:, 1])
plt.title("Conjunto de Dados Ruspini")
plt.show()
```



1.2 4. Aplicando o K-Means

O K-Means precisa receber previamente o número de grupos desejado.

O parâmetro mais importante é:

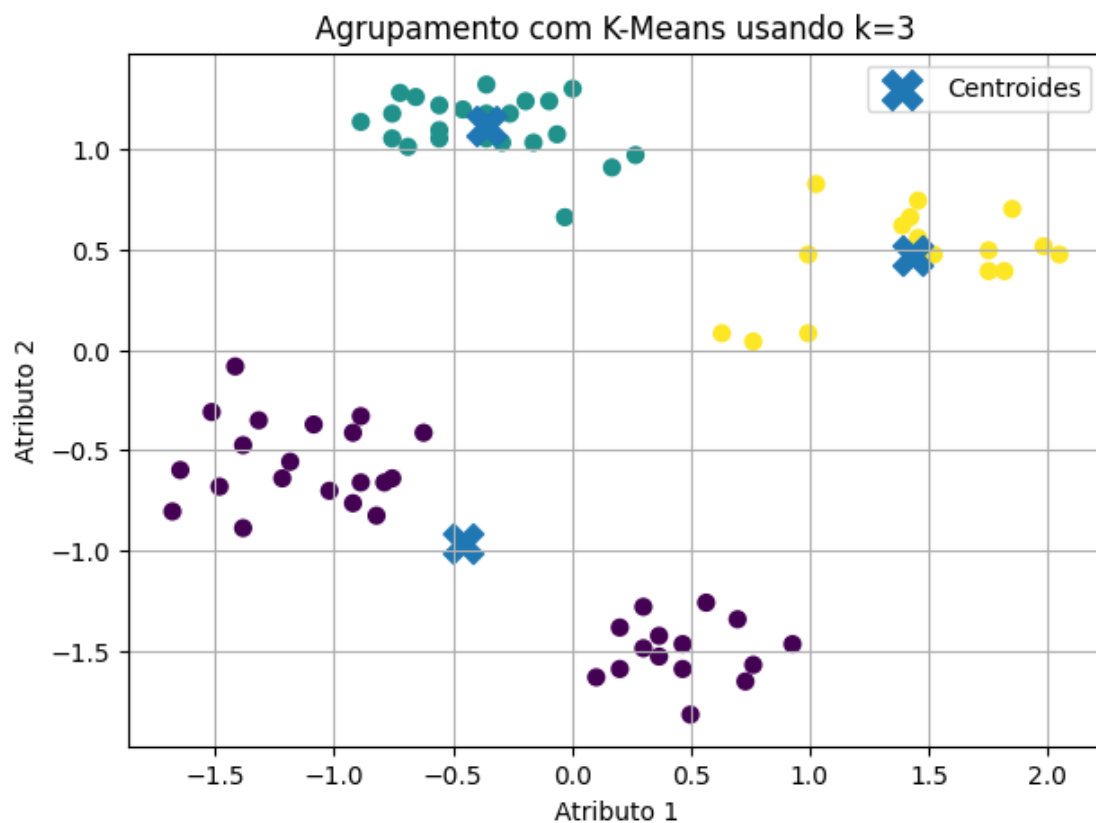
- **n_clusters**: quantidade de grupos que o algoritmo deve encontrar.

Nesta primeira execução, usaremos **n_clusters=3**, porque visualmente os dados parecem ter três grupos.

```
[23]: kmeans = KMeans(
        n_clusters=3,
        random_state=42,
        n_init=10
    )

    labels_kmeans = kmeans.fit_predict(X_scaled)
    centroids = kmeans.cluster_centers_

    plot_clusters(
        X_scaled,
        labels=labels_kmeans,
        centroids=centroids,
        title="Agrupamento com K-Means usando k=3"
    )
```



1.3 5. Como o K-Means funciona?

De forma simplificada, o K-Means executa os seguintes passos:

1. Escolhe inicialmente alguns centroides.

2. Associa cada ponto ao centroide mais próximo.
3. Recalcula os centroides com base nos pontos atribuídos a cada grupo.
4. Repete os passos 2 e 3 até estabilizar.

A ideia é minimizar a distância dos pontos até o centroide do seu grupo.

No scikit-learn, essa medida aparece no atributo `inertia_`.

Quanto menor a inércia, mais próximos os pontos estão dos centroides. Porém, uma inércia menor nem sempre significa melhor modelo, porque aumentar o número de grupos tende naturalmente a reduzir a inércia.

```
[24]: print("Inércia do modelo:", kmeans.inertia_)  
      print("Centroides encontrados:")  
      print(centroids)
```

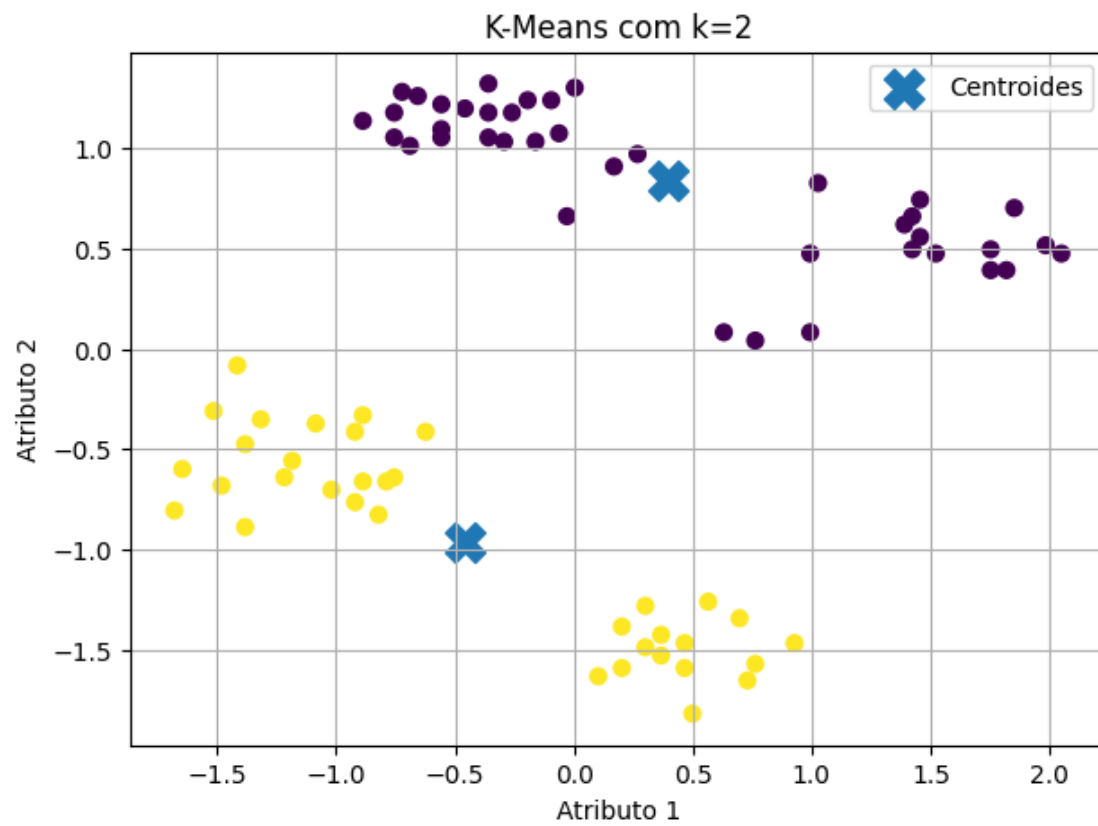
```
Inércia do modelo: 40.04360681955677  
Centroides encontrados:  
[[-0.45622253 -0.96323135]  
 [-0.36196371  1.11658397]  
 [ 1.4289973   0.47245093]]
```

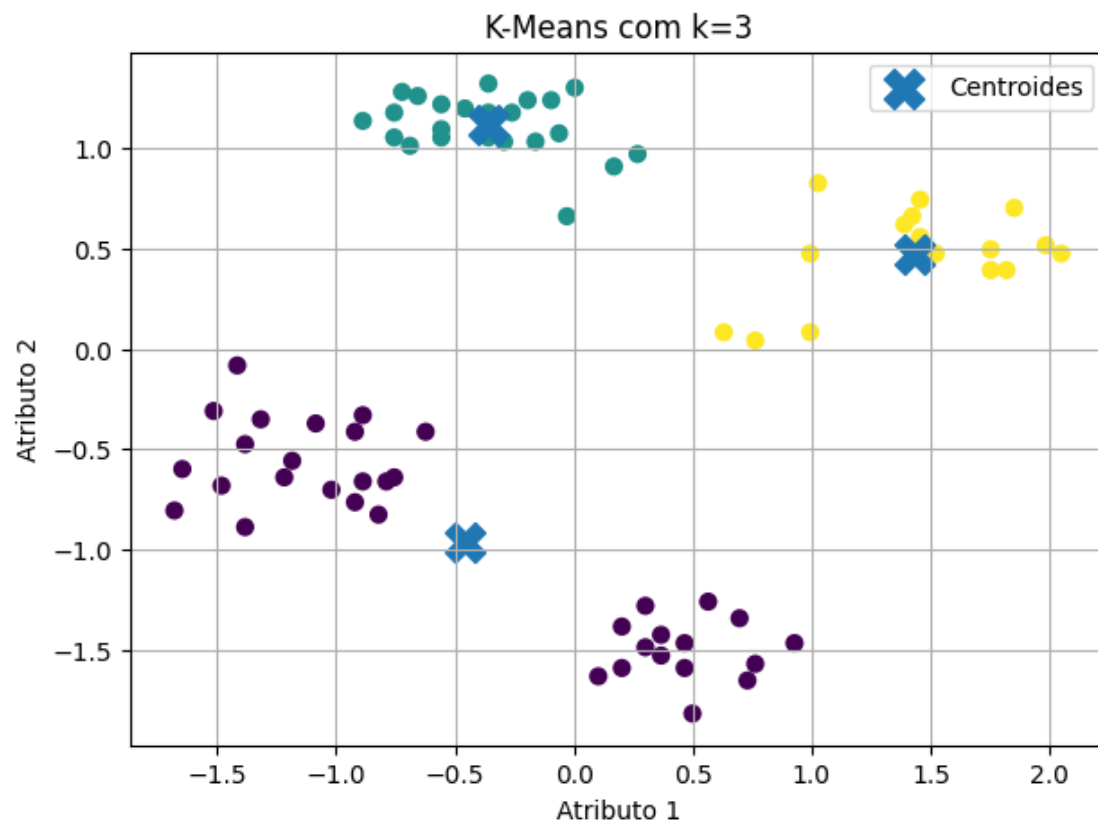
1.4 6. Testando diferentes valores de k

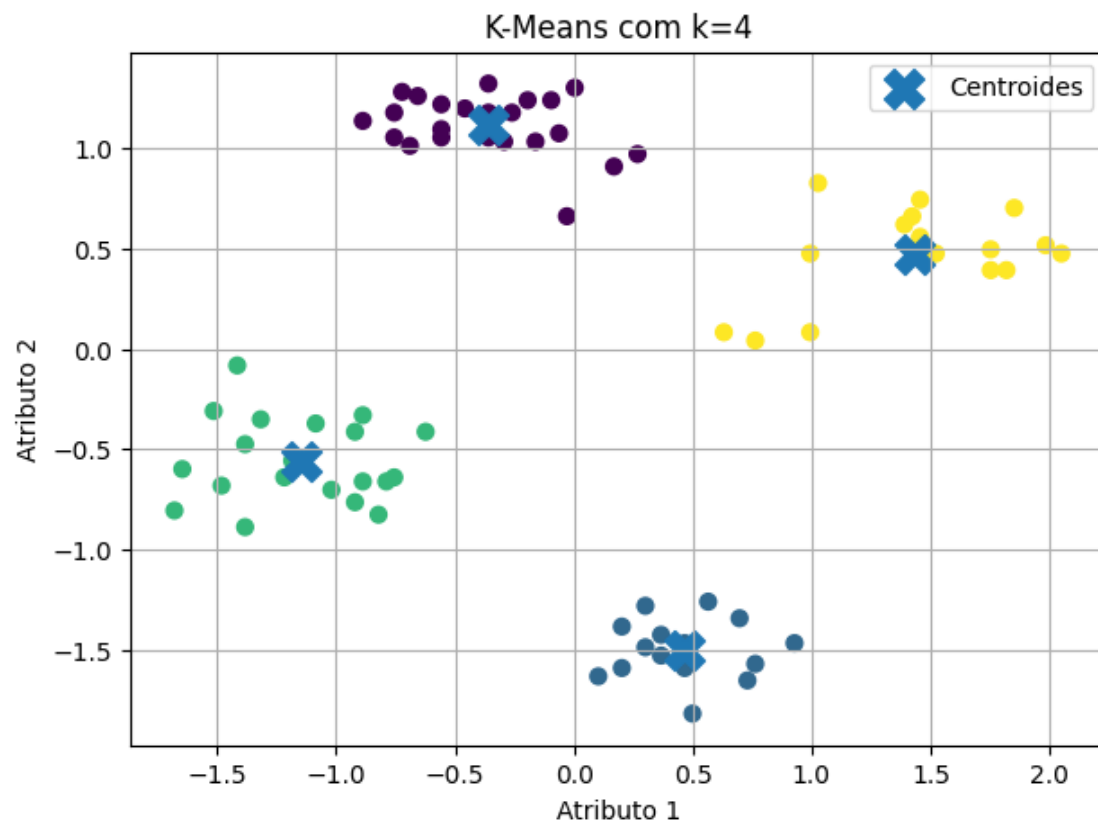
Agora vamos executar o K-Means com diferentes valores de `k`.

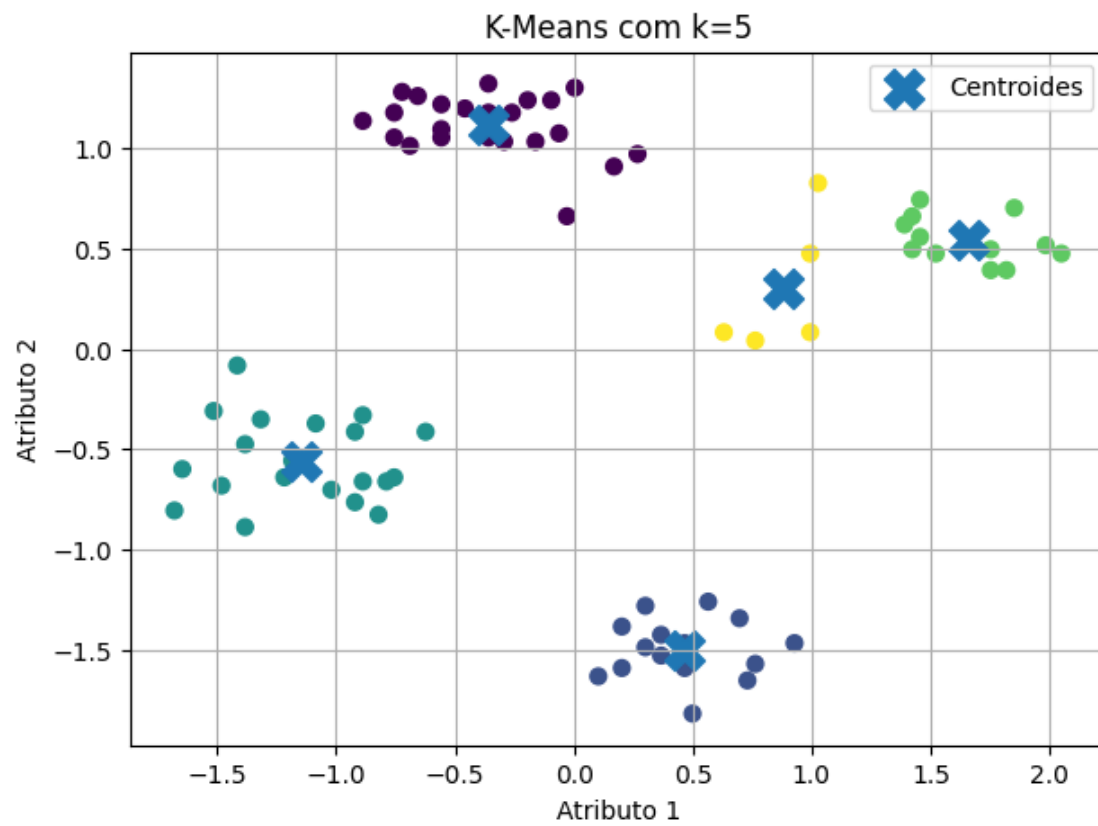
Observe como a escolha de `k` afeta o resultado.

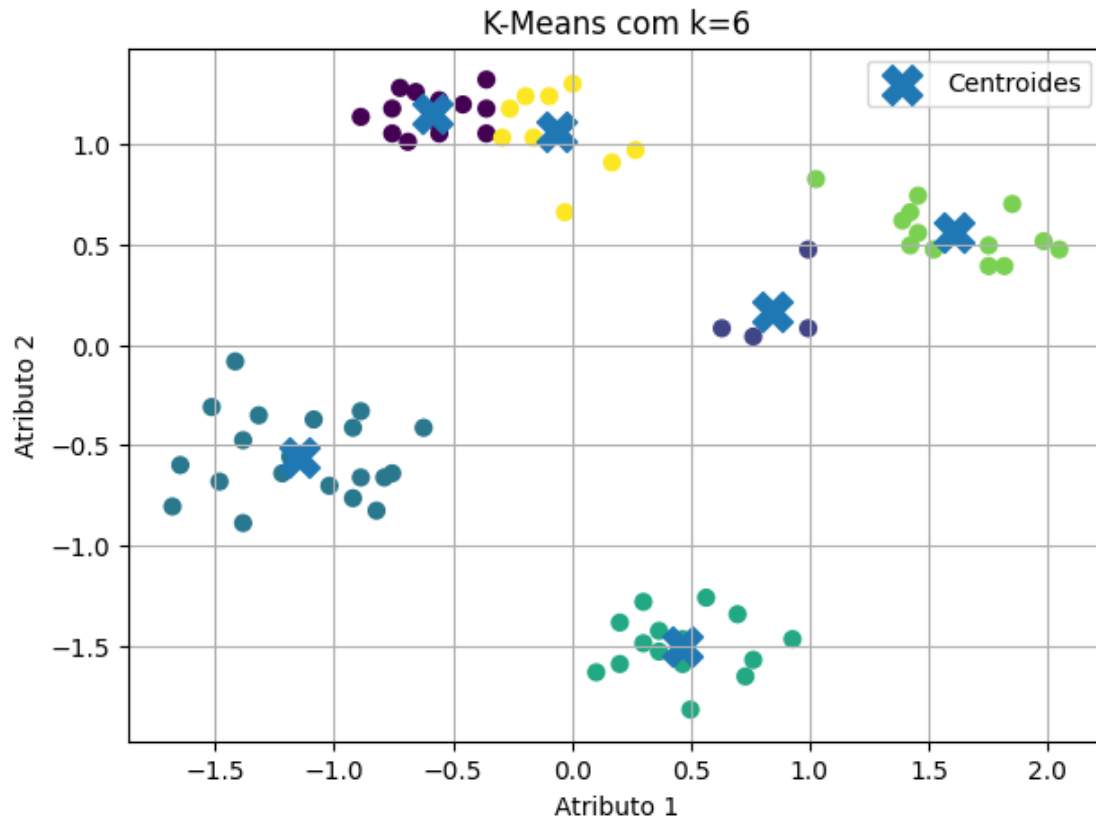
```
[33]: for k in [2, 3, 4, 5, 6]:  
      model = KMeans(n_clusters=k, random_state=42, n_init=10)  
      labels = model.fit_predict(X_scaled)  
  
      plot_clusters(  
          X_scaled,  
          labels=labels,  
          centroids=model.cluster_centers_,  
          title=f"K-Means com k={k}"  
      )
```











1.5 7. Método do cotovelo

Uma forma comum de escolher o valor de k é o **método do cotovelo**.

A ideia é executar o K-Means com diferentes valores de k e observar a inércia.

Quando o ganho começa a diminuir muito, temos uma indicação de um possível valor adequado de k .

Esse ponto é chamado de “cotovelo” do gráfico.

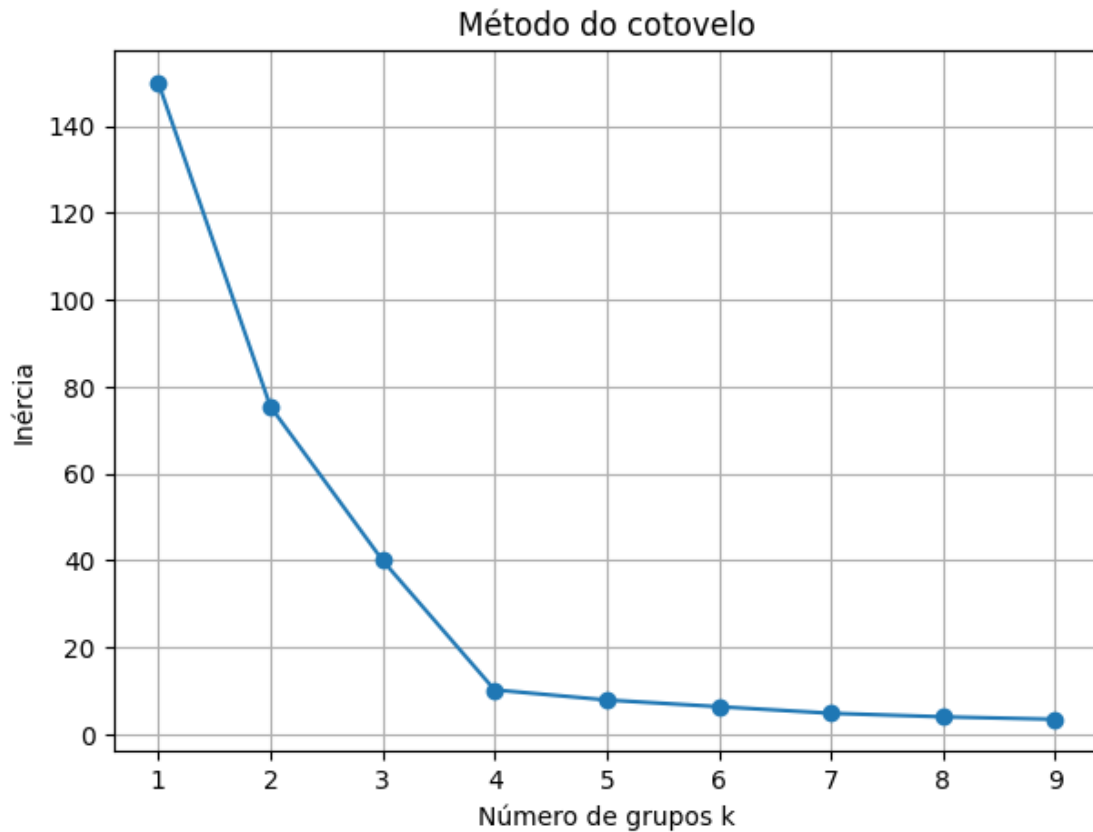
```
[25]: inertias = []

k_values = range(1, 10)

for k in k_values:
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X_scaled)
    inertias.append(model.inertia_)

plt.figure(figsize=(7, 5))
plt.plot(k_values, inertias, marker="o")
```

```
plt.title("Método do cotovelo")
plt.xlabel("Número de grupos k")
plt.ylabel("Inércia")
plt.grid(True)
plt.show()
```



1.6 8. Coeficiente de silhueta

Outra forma de avaliar agrupamentos é o **coeficiente de silhueta**.

Ele mede, de forma simplificada:

- Se os pontos estão próximos dos pontos do próprio grupo.
- Se estão distantes dos pontos de outros grupos.

O valor varia aproximadamente entre -1 e 1.

- Próximo de 1: agrupamento bem separado.
- Próximo de 0: grupos sobrepostos ou pouco definidos.
- Negativo: pontos possivelmente atribuídos ao grupo errado.

Atenção: essa métrica só pode ser calculada quando há pelo menos dois grupos.

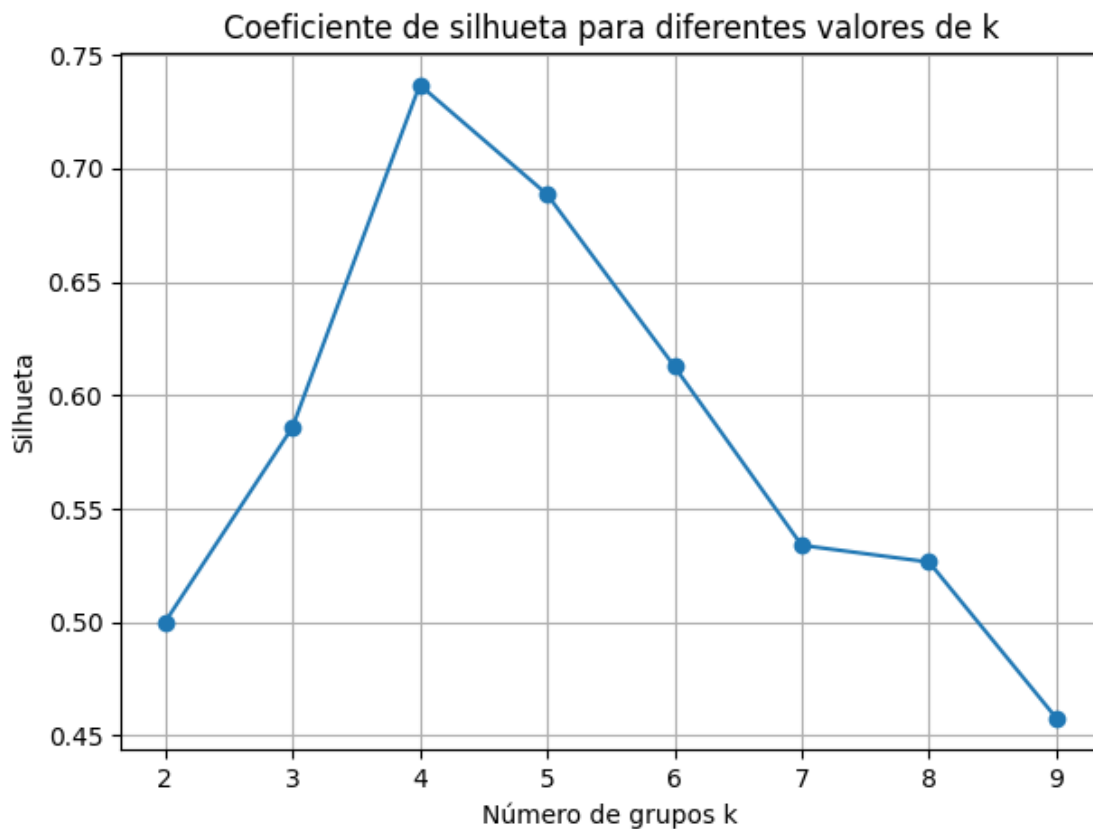
```
[26]: silhouette_scores = []

k_values = range(2, 10)

for k in k_values:
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = model.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    silhouette_scores.append(score)

plt.figure(figsize=(7, 5))
plt.plot(k_values, silhouette_scores, marker="o")
plt.title("Coeficiente de silhueta para diferentes valores de k")
plt.xlabel("Número de grupos k")
plt.ylabel("Silhueta")
plt.grid(True)
plt.show()

for k, score in zip(k_values, silhouette_scores):
    print(f"k={k} | silhueta={score:.3f}")
```



```
k=2 | silhueta=0.500
k=3 | silhueta=0.586
k=4 | silhueta=0.737
k=5 | silhueta=0.689
k=6 | silhueta=0.613
k=7 | silhueta=0.534
k=8 | silhueta=0.526
k=9 | silhueta=0.458
```

2 Parte 2 — Limitações do K-Means

2.1 9. Criando dados em formato de luas

Agora vamos criar um conjunto de dados com formato não circular.

Esse tipo de estrutura é mais difícil para o K-Means, porque o algoritmo tende a separar os grupos usando regiões em torno de centroides.

2.2 10. Aplicando K-Means nos dados em formato de luas

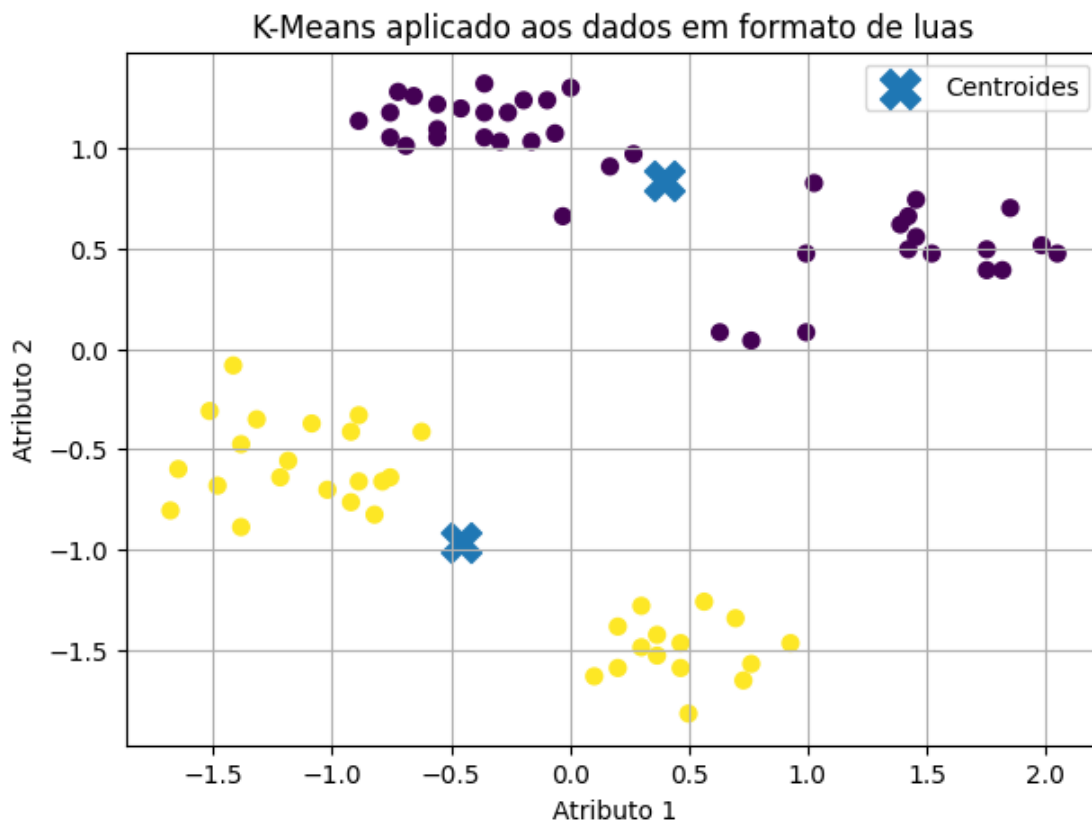
Visualmente, existem dois grupos em forma de meia-lua.

Vamos verificar o que acontece quando pedimos ao K-Means para encontrar dois grupos.

```
[28]: kmeans_moons = KMeans(
        n_clusters=2,
        random_state=42,
        n_init=10
    )

    labels_kmeans_moons = kmeans_moons.fit_predict(X_scaled)

    plot_clusters(
        X_scaled,
        labels=labels_kmeans_moons,
        centroids=kmeans_moons.cluster_centers_,
        title="K-Means aplicado aos dados em formato de luas"
    )
```



O K-Means pode não separar bem grupos com formatos complexos.

Isso acontece porque ele representa cada grupo por um centroide. Assim, grupos alongados, curvos ou com formatos não convexos podem ser divididos de forma inadequada.

Agora vamos usar um algoritmo baseado em densidade: o DBSCAN.

3 Parte 3 — DBSCAN

3.1 11. Intuição do DBSCAN

O DBSCAN agrupa pontos com base em densidade.

Ele utiliza principalmente dois parâmetros:

- **eps**: distância máxima para considerar pontos como vizinhos.
- **min_samples**: número mínimo de pontos necessários para formar uma região densa.

O DBSCAN classifica os pontos em três ideias principais:

1. **Ponto central**: tem vizinhos suficientes dentro do raio **eps**.
2. **Ponto de borda**: está perto de um ponto central, mas não possui vizinhos suficientes.
3. **Ruído/outlier**: não pertence a nenhum grupo.

Uma vantagem importante do DBSCAN é que ele não exige informar previamente o número de grupos.

3.2 12. Aplicando DBSCAN

Agora vamos aplicar o DBSCAN aos dados em formato de luas.

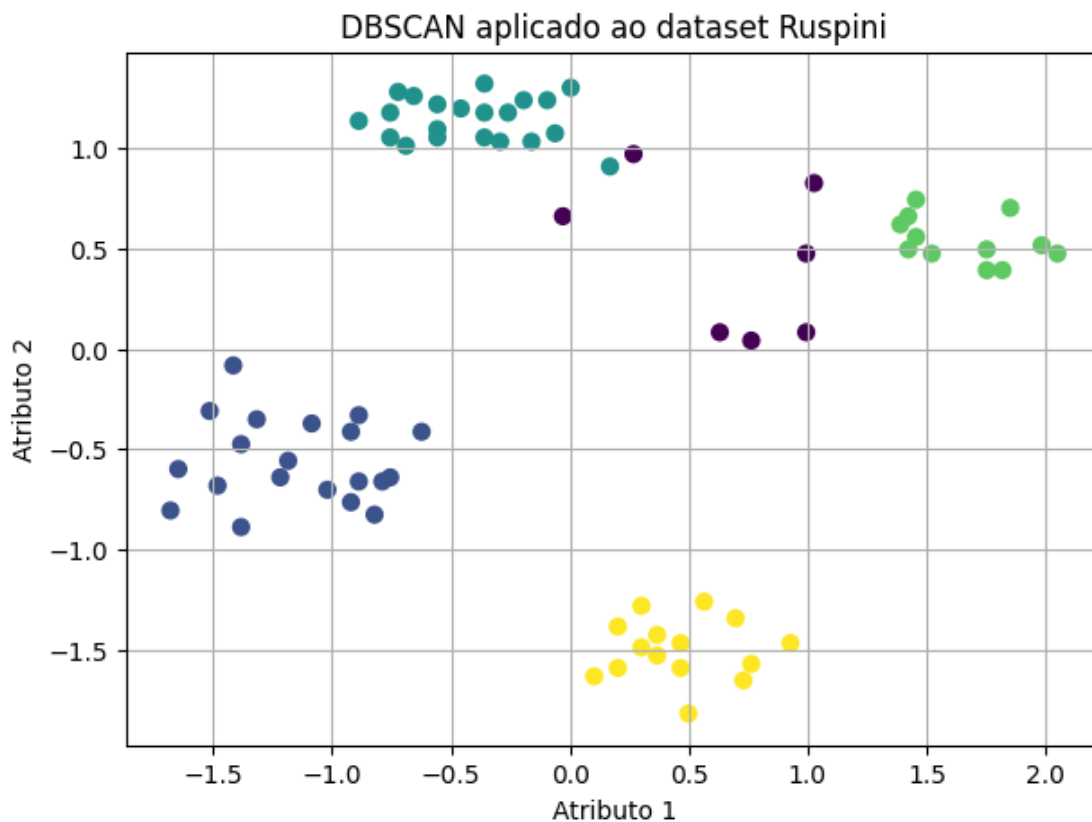
Observe que não informamos o número de grupos. O algoritmo tentará encontrar os grupos com base na densidade dos pontos.

```
[29]: dbscan_ruspini = DBSCAN(
        eps=0.3,
        min_samples=5
    )

    labels_dbscan_ruspini = dbscan_ruspini.fit_predict(X_scaled)

    plot_clusters(
        X_scaled,
        labels=labels_dbscan_ruspini,
        title="DBSCAN aplicado ao dataset Ruspini"
    )

    print("Rótulos encontrados pelo DBSCAN:", np.unique(labels_dbscan_ruspini))
    print("Quantidade de pontos classificados como ruído:", np.
        ↪sum(labels_dbscan_ruspini == -1))
```



Rótulos encontrados pelo DBSCAN: [-1 0 1 2 3]
 Quantidade de pontos classificados como ruído: 7

3.3 13. Interpretando os rótulos do DBSCAN

No DBSCAN, o rótulo -1 representa ruído ou outlier.

Exemplo:

```
labels == -1
```

indica quais pontos foram considerados ruído.

Os demais rótulos, como 0, 1, 2, representam os grupos encontrados.

```
[30]: # Separando visualmente apenas os pontos considerados ruído

noise_mask = labels_dbscan_ruspini == -1

print("Total de pontos:", len(labels_dbscan_ruspini))
print("Pontos em grupos:", np.sum(~noise_mask))
print("Pontos como ruído:", np.sum(noise_mask))
```


Total de pontos: 75
Pontos em grupos: 68
Pontos como ruído: 7

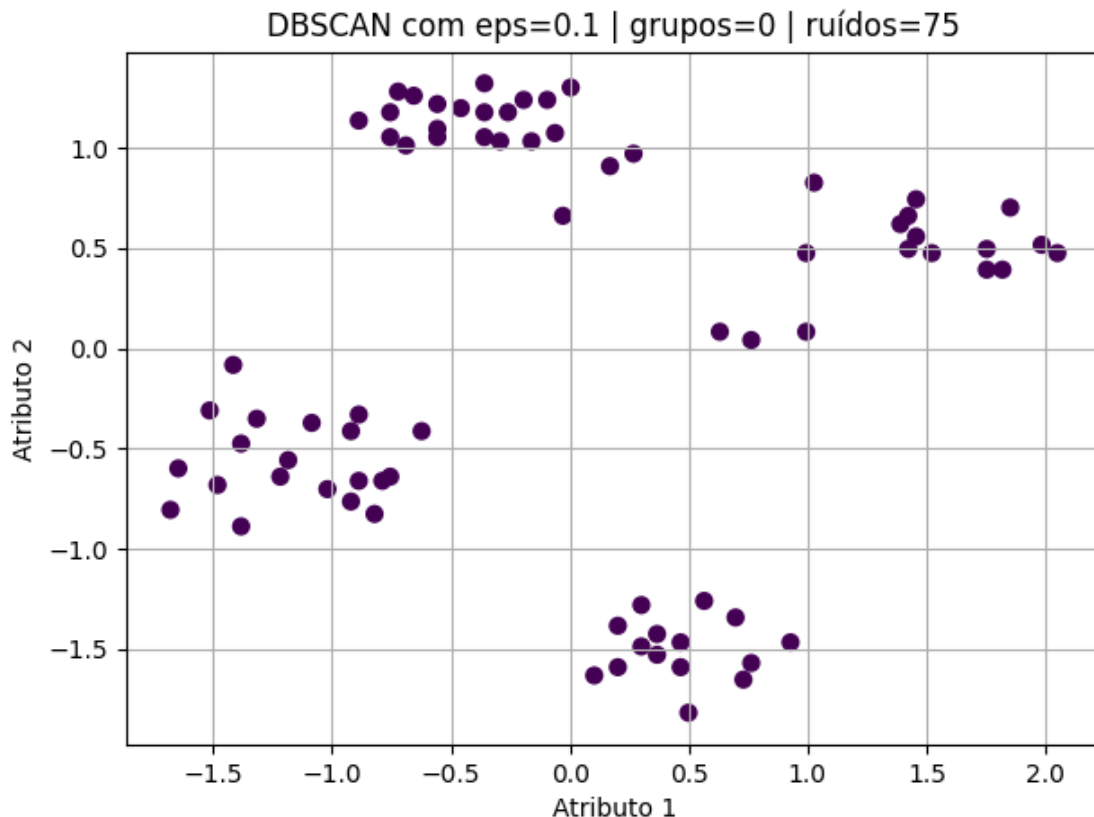
3.4 14. Sensibilidade ao parâmetro eps

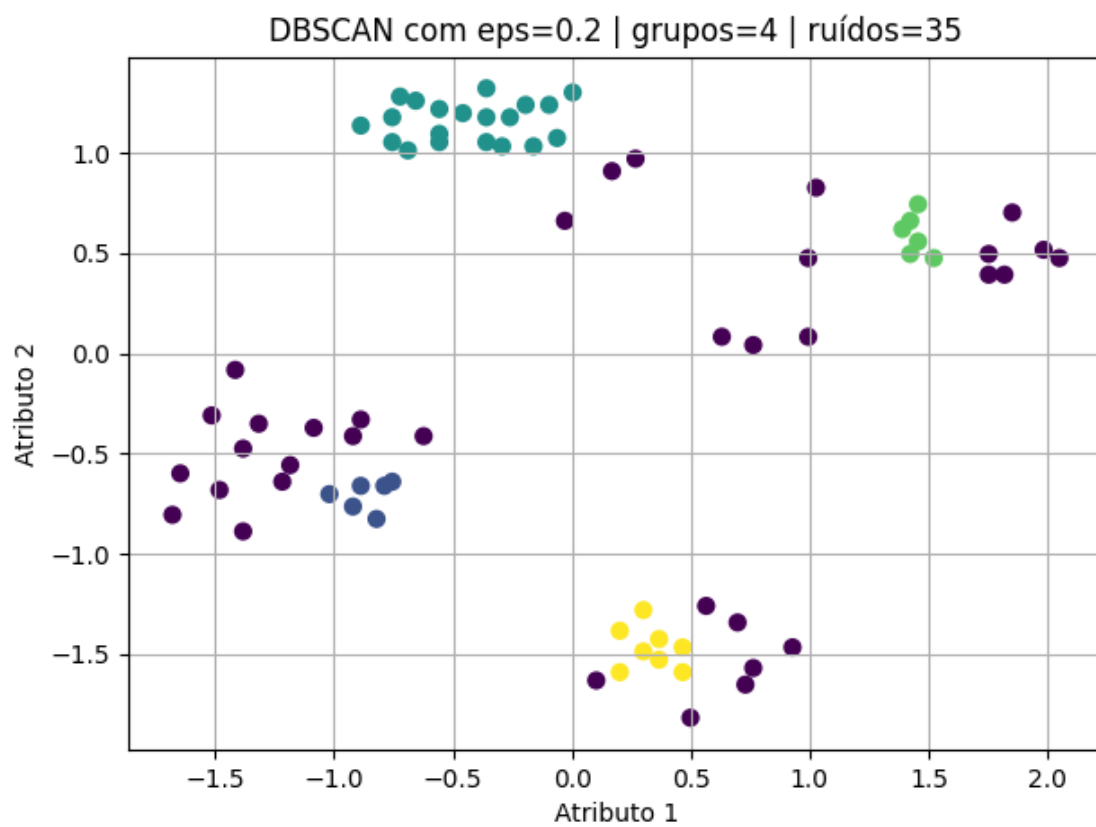
O parâmetro `eps` é muito importante.

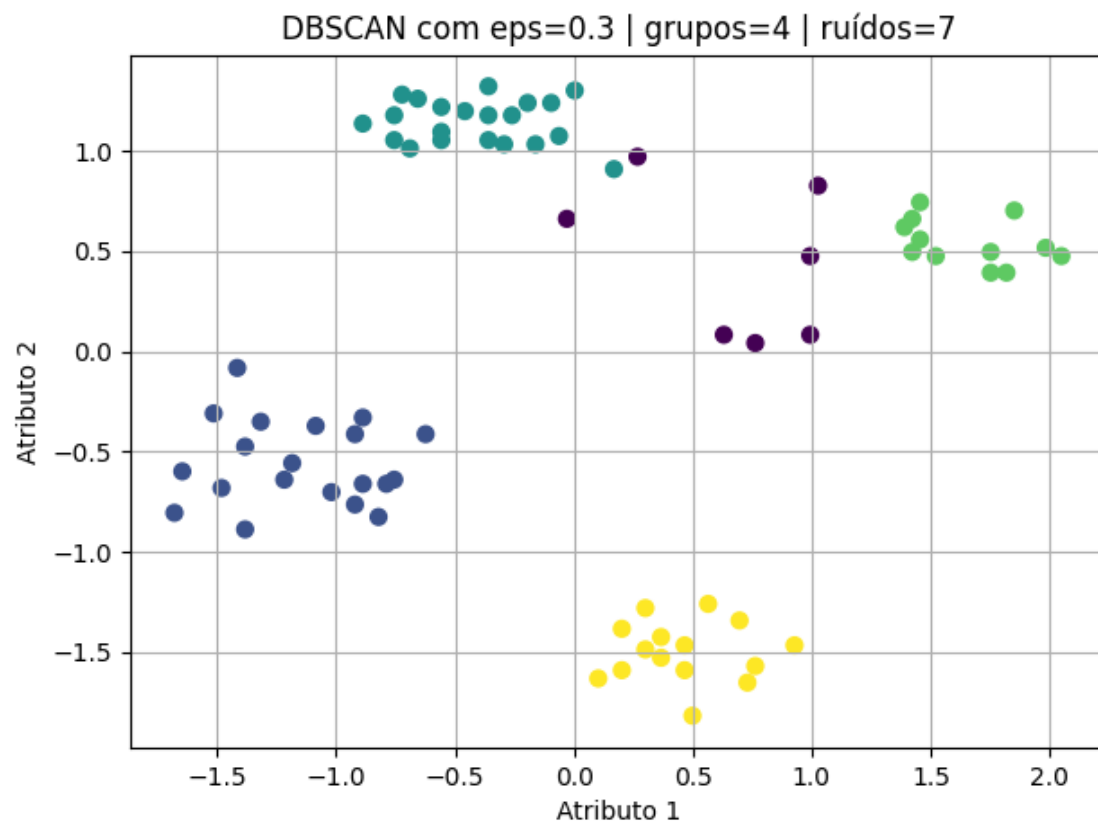
- Se `eps` for muito pequeno, muitos pontos podem virar ruído.
- Se `eps` for muito grande, grupos diferentes podem ser unidos em um único grupo.

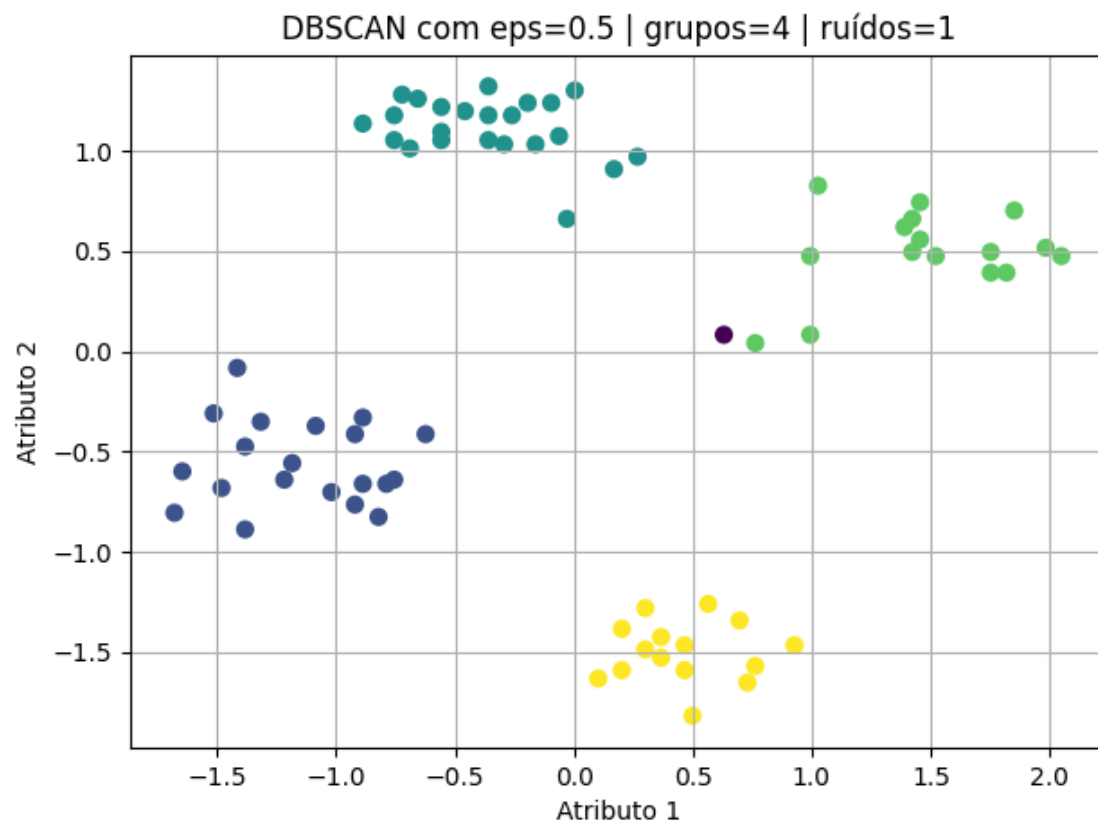
Vamos testar diferentes valores.

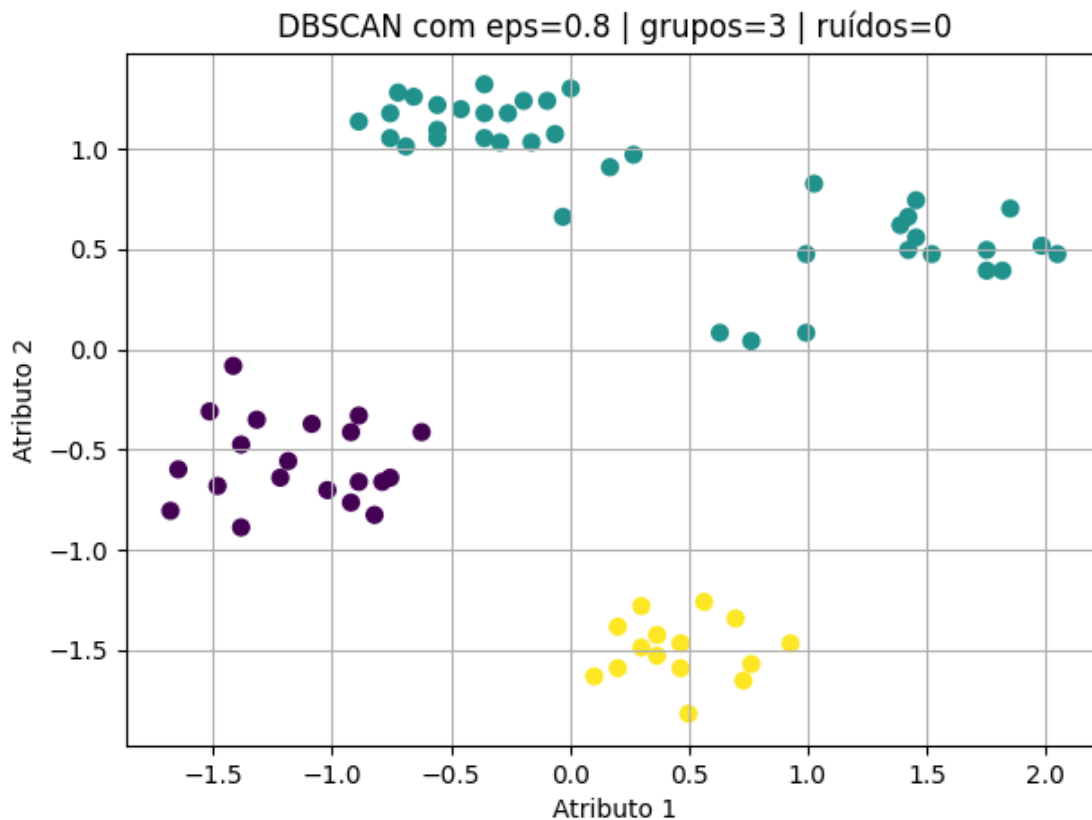
```
[31]: for eps in [0.1, 0.2, 0.3, 0.5, 0.8]:  
      model = DBSCAN(eps=eps, min_samples=5)  
      labels = model.fit_predict(X_scaled)  
  
      n_clusters = len(set(labels)) - (1 if -1 in labels else 0)  
      n_noise = np.sum(labels == -1)  
  
      plot_clusters(  
          X_scaled,  
          labels=labels,  
          title=f"DBSCAN com eps={eps} | grupos={n_clusters} | ruídos={n_noise}"  
      )
```











4 Parte 4 — DBSCAN com ruídos

4.1 15. Criando dados com ruído

Agora vamos criar grupos e adicionar pontos aleatórios espalhados.

A ideia é mostrar uma vantagem do DBSCAN: identificar pontos que não pertencem claramente a nenhum grupo.

4.2 16. Comparando com K-Means nos dados com ruído

O K-Means sempre tenta colocar todos os pontos em algum grupo.

Ele não possui, por padrão, uma categoria de “ruído”.

Por isso, pontos muito distantes também serão forçados a pertencer a algum centroide.

5 Parte 5 — Comparação conceitual

5.1 K-Means versus DBSCAN

Critério	K-Means	DBSCAN
Tipo de abordagem	Baseada em centroides	Baseada em densidade
Precisa informar o número de grupos?	Sim	Não
Lida bem com grupos circulares?	Sim	Sim
Lida bem com grupos curvos/não convexos?	Geralmente não	Sim, em muitos casos
Identifica ruídos/outliers?	Não, por padrão	Sim
Sensível à escala dos dados?	Sim	Sim
Parâmetros principais	<code>n_clusters</code>	<code>eps</code> , <code>min_samples</code>
Dificuldade prática	Escolher <code>k</code>	Escolher <code>eps</code> e <code>min_samples</code>

5.2 Quando usar K-Means?

Use K-Means quando:

- Você tem uma ideia aproximada do número de grupos.
- Os grupos parecem compactos e aproximadamente circulares.
- Você quer um algoritmo simples, rápido e fácil de explicar.
- Não há muitos outliers.

5.3 Quando usar DBSCAN?

Use DBSCAN quando:

- Você não sabe o número de grupos.
- Os dados podem ter ruídos/outliers.
- Os grupos podem ter formatos não circulares.
- Você deseja encontrar regiões densas nos dados.

6 Comentários

Se tratando do kmeans o melhor dos `k` foi 4, isso é comprovado no método do cotovelo e também ao calcular a silhueta, que se obtém o maior valor `k=4` | silhueta=0.737 Se tratando do DBSCAN ao meu ver o `eps` 0.5 e 0.4 obtiveram os melhores valores pois ele conseguiu agrupar os 4 conjuntos e o primeiro teve apenas 1 ponto de ruído enquanto o segundo 7. A partir de `eps` acima de 0.5 ele diminuiu o número de conjuntos.